

Spec-Driven Development, Spec-as-SSOT, Conductor & OpenSpec

The Concise Practical Field Guide: Workflows, Commands, Playbooks & Copy-Pasteable Templates

What Is Inside This Concise Field Guide (5 Focused Sections)

- **Section 1 — Why SDD, Spec-as-SSOT & 5-Minute Quickstart:** Escaping “Vibe Coding”, the 5-minute install & init commands, Spec-First vs. Spec-Anchored vs. Spec-as-SSOT, RFC 2119 (SHALL/MUST) + Gherkin (WHEN/THEN), and when to use Conductor vs. OpenSpec vs. SPEC.md.
- **Section 2 — Conductor Hands-On Guide (“Measure Twice, Code Once”):** The 5 core capabilities, conductor/ directory layout, the 3-step daily loop (/conductor:setup → /conductor:newTrack → /conductor:implement), Red-Green-Refactor TDD with git notes and [checkpoint: <sha>], and Day-2 commands (status, review, revert).
- **Section 3 — OpenSpec Hands-On Guide (The “Core 4” & Delta Specs):** Two-plane directory layout (openspec/specs/ vs. openspec/changes/), /openspec-* vs. /opsx:* naming, the 99% vs. 1% CLI rule, the “Core 4” workflow (explore → propose → apply → archive), and a complete change bundle (ADDED, MODIFIED, REMOVED, RENAMED).
- **Section 4 — OpenSpec Playbooks, Stacked PRs (git / jj) & Security Guardrails:** The 4 scenario playbooks, mapping 1 Change = 1 Feature across Single-PR, Two-Stage Spec-First PR, and Stacked Multi-PRs (tasks.md Batches + Dedicated Archive PR N+1), and the 5 mandatory Golden Rules.
- **Section 5 — Spec-as-SSOT (SPEC.md) Rebuild Test & All 11 OpenSpec Skills Reference:** The 8-section SPEC.md blueprint, 3-Phase Clean-Room Rebuild Test, Cognitive Layer vs. Deterministic CLI Engine architecture, and the complete 11-skill reference table with session outputs.

Contents

1	Why SDD, Spec-as-SSOT & 5-Minute Quickstart	2
1.1	The 3 Levels of SDD Maturity & Choosing the Right Tool	3
2	Conductor Hands-On Guide: “Measure Twice, Code Once”	5
2.1	The 3-Step Daily Workflow (setup → newTrack → implement)	5
2.2	Day-2 Operations: Stack Lock, /conductor:status, review & revert	7
2.3	Dual-Engine Wiring: Combining Conductor with Dedicated Spec Skills & OpenSpec	7
3	OpenSpec Hands-On Guide: The “Core 4” & Delta Specifications	8
3.1	Slash-Command Naming, The 99% vs. 1% Rule & The “Core 4” Workflow	8

4	OpenSpec Playbooks, Stacked PR Workflows & Security Guardrails	10
4.1	The 4 Scenario-Based OpenSpec Playbooks	10
4.2	Version Control & Stacked PR Workflows (1 Change = 1 Feature)	11
5	Spec-as-SSOT (SPEC.md) & Complete Command Reference	12
5.1	Authoring Rebuildable SPEC.md Blueprints & The Rebuild Test	12
5.2	Why OpenSpec Has Both Agent Skills and a CLI Binary	13
5.3	Complete Reference for All 11 OpenSpec Skills & CLI Commands	14
5.4	Curated Public Codelabs, Video Walkthroughs & Courses	14

1 Why SDD, Spec-as-SSOT & 5-Minute Quickstart

Why Spec-Driven Development? (Escaping “Vibe Coding”). When you ask an AI coding assistant in a chat window to “*add webhook retries*” or “*refactor the auth middleware*” without a written specification, the agent guesses your architecture from whatever files happen to be in its context window. In production repositories, unconstrained conversational prompting (“**Vibe Coding**”) consistently breaks down in four ways illustrated in Figure 1 below:

- **Silent Architectural Drift:** The agent invents new database tables, installs unapproved packages, or bypasses existing service layers because no file defines your repository’s boundaries.
- **Unstated Edge Cases:** Chat prompts describe the happy path; rate limits, replay attacks, and rollback behavior are left to the model’s imagination.
- **Painful Code Reviews:** Teammates are forced to reverse-engineer what you *intended* to build by reading 1,500 lines of AI-generated diffs.
- **Context Amnesia:** The moment you close your terminal or start a new chat session, the architectural rationale behind yesterday’s prompt vanishes.

5-Minute Terminal Quickstart. You can install and initialize both **Conductor** (for repository context, TDD track execution, and Git checkpoints) and **OpenSpec** (for capability-level delta specifications and RFC 2119 validation) in under two minutes:

5-Minute Terminal Quickstart: Install & Initialize Conductor and OpenSpec

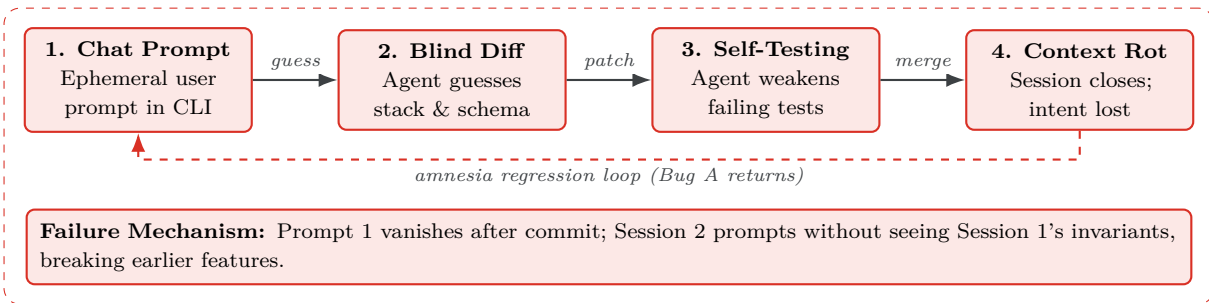
```
# 1. Install Conductor extension for Gemini CLI
$ gemini extensions install https://github.com/gemini-cli-extensions/conductor --auto-update

# 2. Install OpenSpec CLI globally (requires Node.js >= 20.19.0)
$ npm install -g @fission-ai/openspec@latest

# 3. Initialize OpenSpec in your repository (or sub-package in a monorepo)
$ cd ~/projects/order-pulse && openspec init
# Creates openspec/config.yaml, openspec/specs/, and openspec/changes/

# 4. Launch Gemini CLI and initialize Conductor repository context
$ gemini
> /conductor:setup
# Generates conductor/{index,product,product-guidelines,tech-stack,workflow,tracks}.md
```

A. The Conversational “Vibe Coding” Amnesia Loop (Unconstrained Agent Execution)



B. The Context- & Spec-Driven Engineering Loop (Deterministic Governance & Living Sync)

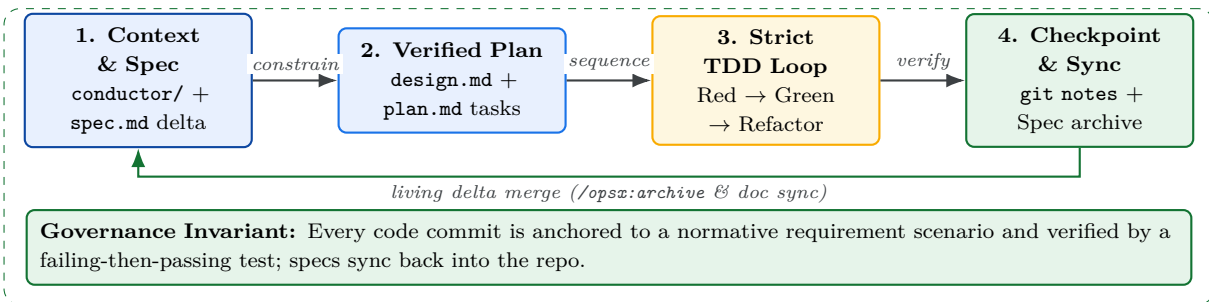


Figure 1: Contrast between the “Vibe Coding” Amnesia Loop (A, top) and the Context- and Spec-Driven Engineering Loop (B, bottom).

```

# 5. Propose a capability spec delta (OpenSpec) or run a TDD feature track (Conductor)
> /openspec-propose "Enforce X-Idempotency-Key on POST /v1/orders"
> /conductor:newTrack "Implement POST /v1/orders idempotency key enforcement"
> /conductor:implement

```

1.1 The 3 Levels of SDD Maturity & Choosing the Right Tool

Spec-First vs. Spec-Anchored vs. Spec-as-Source (SSOT). Teams adopt Spec-Driven Development across three practical maturity levels summarized in Table 1:

- Level 1 — Spec-First (Disposable Planning):** You write a spec or plan before coding a feature, use it to guide the AI agent for that single PR, and then let the spec go stale. Useful for one-off spikes, but future sessions still suffer from context rot.
- Level 2 — Spec-Anchored (Living Repository Specs):** Specifications live permanently in Git alongside your code (`openspec/specs/` and `conductor/`). Every feature branch updates the spec delta and archives it back into the main spec tree upon completion so specs and code never drift.
- Level 3 — Spec-as-Single-Source-of-Truth (Spec-as-SSOT / `SPEC.md`):** An 8-section canonical `SPEC.md` file captures all essential system behavior (domain intent, data schemas, API contracts, state machines, and parameterized Terraform) with enough precision to pass **The Rebuild Test**—regenerating a working, test-passing service from `SPEC.md` alone in an empty directory.

Why Specifications Use ALL-CAPS (SHALL, MUST) and Gherkin (WHEN / THEN). Both OpenSpec and `SPEC.md` use two industry standards to eliminate natural-language ambiguity:

- IETF RFC 2119 & RFC 8174 ALL-CAPS Keywords (SHALL, MUST, SHOULD, MAY):** Casual prose (“returns 429 on rate limit”) is ambiguous. Following RFC 2119 and RFC 8174 (which specifies that

Tool / Paradigm	Files on Disk	Core Strength	When to Use It
Conductor (conductor/)	product.md, tech-stack.md, workflow.md, tracks/<id>/ {spec,plan}.md	Project-wide context lock, strict Red-Green-Refactor TDD (>80% cov), 7-char commit SHAs, git notes, /conductor:revert.	Daily feature & bug implementation where strict TDD, stack governance, and clean Git rollback matter most.
OpenSpec (openspec/)	specs/<cap>/spec.md + changes/<name>/ {proposal, specs, design, tasks}	Two-plane delta specs (ADDED, MODIFIED, REMOVED, RENAMED), RFC 2119 + Gherkin validation (validate --strict).	Features touching domain APIs/schemas where reviewing a crisp behavioral spec diff before coding is critical.
Spec-as-SSOT (SPEC.md)	Root 8-Section SPEC.md blueprint	Complete system DDL, API contracts, parameterized IaC, and 3-Phase Clean-Room Rebuild Test.	Extracting a reusable blueprint from a brownfield repo, cross-language rewrites, or greenfield service design.

Table 1: Practical comparison of Conductor, OpenSpec, and Spec-as-SSOT (SPEC.md).

keywords are normative *only* in uppercase), SHALL and MUST define hard invariants that fail verification if violated, SHOULD defines recommended defaults requiring justification in design.md to bypass, and MAY defines optional behavior.

- **Gherkin Scenarios** (GIVEN / WHEN / THEN / AND): Every requirement is paired with concrete #### Scenario: blocks that map 1-to-1 to unit/integration tests: GIVEN sets up test fixtures, WHEN triggers the action, and THEN asserts the exact HTTP status, database state, or output payload.

The Generation–Verification Loop, Human Interaction Gates & Karpathy’s 4 Agent Rules.

In **Software 3.0**, code generation is virtually free while **human verification is the true bottleneck**. To keep coding agents on a “short leash” (the **Autonomy Slider**), Conductor and OpenSpec enforce four behavioral rules and an immutable verification harness (mirroring Karpathy’s autoresearch separation between human-steered program.md and read-only evaluator prepare.py):

1. **Think Before Coding (Plan Mode Q&A Gate)**: Never guess silently on ambiguous requirements; pause in /conductor:newTrack or /opsx:explore to surface trade-offs and ask 3–5 clarifying questions before touching code.
2. **Simplicity First (Spec-First Diff Review)**: Humans review and approve a 50-line behavioral contract (spec.md + ADRs) before implementation, and the agent writes only the minimal code needed to turn failing tests green.
3. **Surgical Changes (Out-of-Scope & Tech-Stack Locks)**: Lock non-target findings into ## 6. Out of Scope and halt on any unapproved dependency not listed in conductor/tech-stack.md.
4. **Goal-Driven Execution & Immutable Verifiers (chmod -w)**: Anchor every task to a 1-to-1 test_reqXXXX_* unit test and phase checkpoint, and lock adversarial verification suites read-only (chmod -w) during Clean-Room Rebuild Tests so an agent can never pass by weakening test assertions.

Joel Spolsky’s Law of Leaky Abstractions (Why Code Generation Saves Time *Working*, Not Time *Learning*). In his classic 2002 essay “*The Law of Leaky Abstractions*” (*Joel on Software*), Joel Spolsky formulated the foundational law that governs every high-level abstraction—and anticipates the central trap of conversational “Vibe Coding” with AI agents:

System Design Invariant & Tie-Breaker

Joel Spolsky's Law of Leaky Abstractions (2002) Applied to AI Code Generation **"All non-trivial abstractions, to some degree, are leaky."**

*"The law of leaky abstractions means that whenever somebody comes up with a wizzy new code-generation tool that is supposed to make us all ever-so-efficient, you hear a lot of people saying 'learn how to do it manually first, then use the wizzy tool to save time.' Code generation tools which pretend to abstract out something, like all abstractions, leak, and the only way to deal with the leaks competently is to learn about how the abstractions work and what they are abstracting. **So the abstractions save us time working, but they don't save us time learning.**"* — Joel Spolsky, *Joel on Software* (Nov 11, 2002)

Engineering Consequence for SDD: Natural-language prompts are the ultimate leaky abstraction. Prompting *"add a distributed lock"* or *"add a token-bucket rate limiter"* hides GC-pause split-brain races (requiring monotonic fencing tokens), sub-quantum integer truncation drift, and half-open probe storms. **Spec-Driven Development** forces you to explicitly specify and test the exact boundary where the abstraction leaks (REQ-XXXX, ADR-*.md, and ##### Scenario:) before the agent generates code.

2 Conductor Hands-On Guide: "Measure Twice, Code Once"

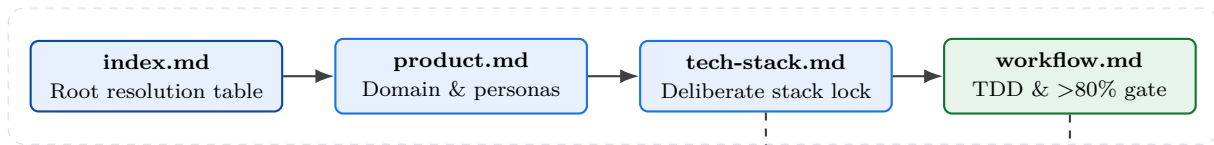
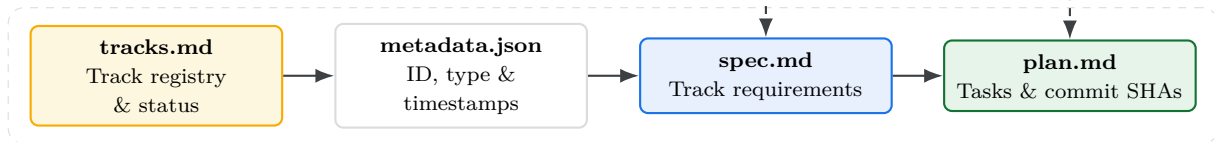
Control Your Code Through Persistent Repository Context. As introduced in the official Google Developers guide (*"Conductor: Introducing context-driven development for Gemini CLI"*, <https://developers.googleblog.com/conductor-introducing-context-driven-development-for-gemini-cli/>), Conductor is built on a simple principle: *"Failing to plan is planning to fail... control your code."* Instead of re-explaining your tech stack, coding conventions, and testing rules in every chat session, Conductor stores project context as version-controlled Markdown artifacts inside `conductor/` (Figure 2 below), unlocking five practical capabilities:

1. **Plan before you build:** Creates explicit specifications (`spec.md`) and phased task plans (`plan.md`) for your approval before any application code is written.
2. **Maintain context:** Automatically enforces your style guides (`conductor/code_styleguides/`), technology choices (`tech-stack.md`), and workflow rules (`workflow.md`) across every session.
3. **Iterate safely:** Lets you inspect and edit concrete Markdown plans before a single source file is touched.
4. **Work as a team:** Commits `conductor/` to Git so every developer and agent session shares the exact same project foundation.
5. **Build on existing ("brownfield") projects:** Automatically scans mature repositories (`pyproject.toml`, `package.json`, `go.mod`) while respecting `.gitignore` and `.geminiignore`.

2.1 The 3-Step Daily Workflow (setup → newTrack → implement)

Step 1: Establish Context (`/conductor:setup`). Run `/conductor:setup` once per repository. Conductor detects whether the repo is greenfield or brownfield, asks at most 4 interactive questions per phase (distinguishing **Additive** multi-select scope questions from **Exclusive** single-select architecture questions), and generates `conductor/{index,product,product-guidelines,tech-stack,workflow,tracks}.md`. If interrupted, re-running `/conductor:setup` automatically resumes at the exact incomplete file without overwriting prior work.

Step 2: Specify and Plan (`/conductor:newTrack`). Run `/conductor:newTrack "<description>"` whenever you start a feature or bug fix. Conductor reads `product.md` and `tech-stack.md`, asks brief clarifying questions,

A. Project-Level Context Plane (conductor/)**B. Track Execution Plane (conductor/tracks/<track_id>/)**

Plan Mode Security Sandbox (policies/conductor.toml): When planning in Plan Mode, Conductor allows file writes ONLY inside `^conductor/[~]*` and blocks `..` traversal and shell redirection (`>`, `| tee`) at priority = 200.

Figure 2: Conductor's two-plane directory architecture separating persistent project context (Plane A) from feature track execution artifacts (Plane B).

and generates a track folder (`conductor/tracks/<track_id>/`) containing `spec.md` (requirements and acceptance criteria), `plan.md` (phased task checklist with automatic phase verification tasks), and `metadata.json`, then pauses for your review.

Step 3: Implement with Strict TDD (`/conductor:implement`). Once you approve `plan.md`, run `/conductor:implement`. Conductor steps through each task sequentially using strict Red-Green-Refactor TDD:

1. Marks the active task `[-]` in `plan.md`.
2. **Red Phase:** Writes failing unit/contract tests first and runs the test suite (`CI=true`) to confirm failure.
3. **Green & Refactor Phase:** Writes the minimal production code to pass tests and verifies `>80%` coverage.
4. **Commit & Audit Trail:** Commits the code, attaches a structured rationale via `git notes add -m "... <sha>`, and marks the task `[x] <7-char-sha>` in `plan.md`.
5. **Phase Checkpoint:** At the end of each phase, runs the full test suite across all modified files, pauses for your manual verification approval, and records `[checkpoint: <7-char-sha>]` on the phase header.

Hands-On Walkthrough: Conductor `spec.md`, `plan.md` & Terminal Execution Trace

```
# === 1. Start a New Feature Track ===
> /conductor:newTrack "Enforce X-Idempotency-Key on POST /v1/orders"
# Creates conductor/tracks/order_idempotency_20261008/{spec.md, plan.md, metadata.json}

# === 2. Generated conductor/tracks/order_idempotency_20261008/spec.md (Excerpt) ===
# Specification: Order Idempotency Enforcement
## Functional Requirements
- FR-1: 'POST /v1/orders' MUST require a valid UUIDv4 'X-Idempotency-Key' header.
- FR-2: Duplicate requests with the same key and SHA-256 payload hash within 24h
      MUST return HTTP 200 OK with the original Order response without double-charging.
- FR-3: Duplicate requests with a mutated payload MUST return HTTP 422.

# === 3. Generated conductor/tracks/order_idempotency_20261008/plan.md (During Execution) ===
# Implementation Plan: Order Idempotency Enforcement
## Phase 1: Schema & Repository Layer [checkpoint: 9f8e7d6]
```

```

- [x] Task 1.1: Add 'idempotency_key' & 'payload_hash' columns to Order model a1b2c3d
- [x] Write failing SQLAlchemy constraint tests in 'tests/test_order_model.py'
- [x] Implement migration and unique index in 'src/models/order.py'
- [x] Task: Conductor - User Manual Verification 'Phase 1: Schema' 9f8e7d6

## Phase 2: FastAPI Endpoint & Conflict Handling
- [~] Task 2.1: Enforce header validation and hash comparison in 'POST /v1/orders'
- [x] Write failing pytest cases for fresh (201), replay (200), and mutated (422)
- [ ] Implement idempotency lookup in 'src/api/orders.py'

# === 4. Inspecting the Attached Git Note for Task 1.1 ===
$ git notes show a1b2c3d
Task 1.1: Add idempotency_key & payload_hash columns to Order model
- Summary: Added nullable=False UUIDv4 key and SHA-256 hash column with unique index.
- Files: src/models/order.py, tests/test_order_model.py
- Verification: CI=true pytest --cov=src (8 passed, 94% coverage)

```

2.2 Day-2 Operations: Stack Lock, /conductor:status, review & revert

Tech-Stack Deviation Gate, Code Review & Logical Git Revert. Table 2 summarizes all six Conductor commands and how they handle mid-track stack changes, automated code reviews, and logical rollbacks.

Command / Rule	How It Works	Practical Behavior & Guardrails
/conductor:setup	Scans repo (respecting .gitignore/.geminiignore) & interviews user.	Creates conductor/*.md. Resumes automatically across 7 state tiers if interrupted.
/conductor:newTrack	Interactive Q&A → generates spec.md, plan.md, metadata.json.	Injects a mandatory User Manual Verification task at the end of every phase in plan.md.
/conductor:implement	Executes [] → [~] → [x] <sha> via Red-Green-Refactor TDD.	Stack Lock Gate: If a task requires a new library not in tech-stack.md, the agent MUST halt, ask permission, and update tech-stack.md first.
/conductor:status	Reads tracks.md and active plan.md files.	Prints completed [x], in-progress [~], pending [] counts, and next actionable task.
/conductor:review	Audits diff against spec.md, tech-stack.md, and code_styleguides/.	Uses unified diff if <300 lines or file-by-file chunking if >300 lines; can auto-apply fixes into a new ## Phase: Review Fixes block in plan.md.
/conductor:revert	4-phase logical Git revert of a Track, Phase, or Task.	Collects implementation SHAs + conductor(plan) SHAs from plan.md (finding rebased “ghost commits” via commit message search) and runs git revert --no-edit in reverse order.

Table 2: Complete reference for Conductor slash commands and execution guardrails.

2.3 Dual-Engine Wiring: Combining Conductor with Dedicated Spec Skills & OpenSpec

Living Guardrails & The 5-Step Brownfield Refactoring Lifecycle. You never need to re-run /conductor:setup to add new team rules or wire OpenSpec and custom spec skills into Conductor: because

conductor/index.md and conductor/workflow.md are re-read at the start of every track creation, implementation, and review cycle, editing them in Git immediately updates agent behavior across your entire team. In production brownfield repositories, combine ****Context-Driven Development (Conductor)**** and ****Spec-Driven Development (OpenSpec / Dedicated Spec Skill)**** via the 5-step refactoring loop and two-file handshake shown below:

Dual-Engine Handshake (conductor/workflow.md + index.md) & 5-Step Refactoring Loop

```
# === 1. Add to conductor/workflow.md (Track Specification Policy) ===
### Track Specification Policy (Dual-Engine CDD + SDD Handshake)
- When executing '/conductor:newTrack', conduct interactive Q&A to clarify scope.
- Upon questioning completion, invoke '@writing-arch-specs'
  ('.agents/skills/writing-arch-specs/SKILL.md') and align with 'openspec/changes/'
  to draft 'conductor/tracks/<track_id>/spec.md' using the 7-Section Anatomy:
  1. Executive Summary, Legacy Diagnosis & Objective
  2. Environment Prerequisites, Dependencies & Tech-Stack Lock
  3. Technical Contracts, Schemas & Architecture
  4. Concrete Deliverables & Target Files
  5. Verification Gates & Acceptance Criteria (RFC 2119 + Gherkin WHEN/THEN)
  6. Cost, Concurrency Trade-Offs & Rollback Safeguards
  7. Explicit 'Out of Scope' Boundaries (prevents drive-by refactoring)

# === 2. Expose in conductor/index.md ===
## Capabilities & Spec Contracts
- [Writing Architecture Specs Skill](../agents/skills/writing-arch-specs/SKILL.md)
- [OpenSpec Capability Contracts](../openspec/specs/)

# === 3. The 5-Step Brownfield Refactoring Workflow ===
Step 1: Review Architecture Findings ('docs/architecture-audit/scorecard.md')
Step 2: Agree on Trade-Offs ('ACCEPTED_FOR_REFACTOR', 'DEFERRED', or 'REJECTED')
Step 3: Draft Spec & TDD Plan via '/conductor:newTrack' (Q&A -> Spec -> 'plan.md')
Step 4: Phased TDD Implementation ('/conductor:implement': Red -> Green -> Commit)
Step 5: Multi-Axis Audit ('/conductor:review' + '/opsx:verify') -> '/opsx:archive'
```

3 OpenSpec Hands-On Guide: The “Core 4” & Delta Specifications

How OpenSpec Organizes Your Repository (Two-Plane Directory Model). When you run `openspec init`, OpenSpec creates an `openspec/` directory that separates **Current System Truth (Plane 1: `openspec/specs/`)** from **Active Change Proposals (Plane 2: `openspec/changes/`)**. Instead of editing master specifications directly while coding, every feature or bug fix lives in its own isolated folder (`openspec/changes/<change-name>/`) containing four Markdown files—`proposal.md`, `specs/<domain>/spec.md`, `design.md`, and `tasks.md`—and is merged into `openspec/specs/` only when you run `/openspec-archive-change` (Figure 3 below). In a **multi-project monorepo**, run `openspec init --force` inside the specific sub-package directory (e.g., `services/order-api/openspec/`) rather than at the monorepo root so `openspec/specs/` stays tightly scoped to that service.

3.1 Slash-Command Naming, The 99% vs. 1% Rule & The “Core 4” Workflow

Why You See Both `/openspec-*` and `/opsx:*` (And the 99% vs. 1% CLI Rule). Upstream OpenSpec docs use colon syntax (`/opsx:propose`, `/opsx:apply`), whereas skill-based AI assistants register skills by directory name (`/openspec-propose`, `/openspec-apply-change`); both invoke the exact same workflow. For

OpenSpec Two-Plane Architecture & Deterministic 5-Stage Delta Merge Pipeline (specs-apply.js)

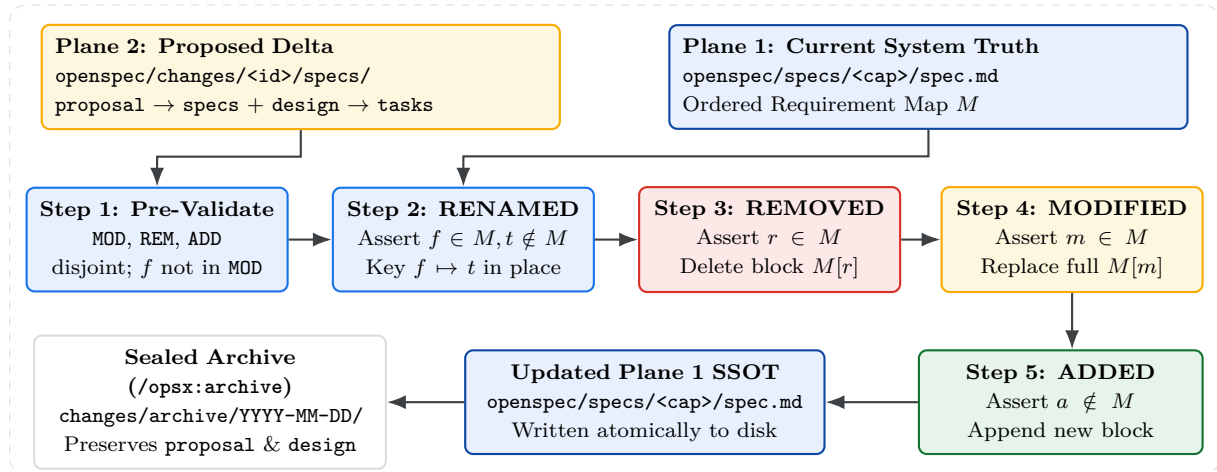


Figure 3: OpenSpec’s Two-Plane directory model and 5-stage delta merge pipeline (Pre-Validate → RENAMED → REMOVED → MODIFIED → ADDED).

99% of daily work, you run **Agent Skills in chat**—the agent calls the `openspec` CLI binary under the hood to scaffold directories and fetch JSON instructions, then writes the Markdown specs and application code. You only run `openspec` directly in your terminal for the remaining **1%**: instant dashboards (`openspec view` or `openspec list`), linting manual spec edits (`openspec validate <name> --strict`), or forking custom schemas (`openspec schema fork`).

Mastering the “Core 4” OpenSpec Workflow (80% of Daily Work). While OpenSpec includes 11 skills (all listed in Section 5), **80% of daily engineering uses just four steps**:

- Think First** (`/openspec-explore / /opsx:explore`): Read-only investigation. The agent reads your codebase, surfaces bottlenecks, and draws ASCII trade-off diagrams without touching code.
- Propose in One Step** (`/openspec-propose / /opsx:propose`): Generates `proposal.md`, `delta specs/`, `design.md`, and `tasks.md` inside `openspec/changes/<name>/` in a single turn.
- The 60-Second Human Sanity Check**: Read `proposal.md`, `specs/`, and `tasks.md` before coding. Fixing a wrong assumption in Markdown takes 10 seconds; fixing 800 lines of code takes hours.
- Execute & Archive** (`/openspec-apply-change → /openspec-archive-change`): Works through `tasks.md` (`- [ ] → - [x]`), then merges deltas into `openspec/specs/` and moves the folder to `changes/archive/YYYY-MM-DD-<name>/`.

Hands-On Walkthrough: Complete OpenSpec Change Bundle (proposal, specs, design, tasks)

```
# === 1. openspec/changes/harden-auth-tokens/proposal.md ===
## Why
Bearer tokens currently remain valid for 60 minutes without refresh rotation,
exposing active sessions if a token is intercepted.
## What Changes
- Add single-use refresh token rotation with replay family revocation.
- Reduce JWT access token TTL from 3600s to 900s.
- **BREAKING** Remove legacy '?api_key=' query-string authentication.
## Capabilities
- Modified Capabilities: 'auth-session'

# === 2. openspec/changes/harden-auth-tokens/specs/auth-session/spec.md ===
# Delta for Auth Session
```

```

## ADDED Requirements
### Requirement: Refresh Token Rotation
The system SHALL issue a single-use refresh token on every exchange and MUST
revoke the entire token family if a consumed refresh token is replayed.
#### Scenario: Replay attack on consumed refresh token
- **GIVEN** refresh token 'rt_01' has already been exchanged for 'rt_02'
- **WHEN** a client submits 'POST /v1/auth/refresh' with 'rt_01'
- **THEN** the system MUST return HTTP 401 and revoke both 'rt_01' and 'rt_02'

## MODIFIED Requirements
### Requirement: JWT Access Token TTL
The system MUST sign access tokens with Ed25519 and enforce a maximum TTL of
900 seconds (reduced from 3600 seconds).
#### Scenario: Expired token rejected after 900 seconds
- **WHEN** a request presents an Ed25519 JWT issued 901 seconds ago
- **THEN** the middleware MUST reject the request with HTTP 401 Unauthorized

## REMOVED Requirements
### Requirement: Legacy Query-String API Key Auth
**Reason**: Query-string keys leak in reverse-proxy access logs.
**Migration**: Pass 'Authorization: Bearer <jwt>' header instead.

## RENAMED Requirements
- FROM: '### Requirement: Token Signature Check'
- TO: '### Requirement: Ed25519 Signature Verification'

# === 3. openspec/changes/harden-auth-tokens/tasks.md ===
## 1. Storage & Middleware Implementation
- [x] 1.1 Add Redis token-family store in 'src/auth/refresh_store.py'
- [x] 1.2 Enforce 900s Ed25519 TTL and remove '?api_key=' in 'src/auth/jwt.py'
- [ ] 1.3 Add pytest replay and TTL expiration tests in 'tests/test_auth.py'

```

4 OpenSpec Playbooks, Stacked PR Workflows & Security Guardrails

4.1 The 4 Scenario-Based OpenSpec Playbooks

Playbook 1 — The “Fast-Track” Daily Driver (Small Features & Bug Fixes, 1–3 Files). When requirements are clear, run `/openspec-propose "<description>" (/opsx:propose)`, perform a **60-Second Sanity Check** on `specs/` and `tasks.md` (checking that edge cases like rate limits or missing headers are covered and no out-of-scope tasks were invented), run `/openspec-apply-change (/opsx:apply)`, and run `/openspec-archive-change (/opsx:archive)`.

Playbook 2 — Deep Architecture & Spec-First PRs (explore → new → continue). When touching unfamiliar code or designing a cross-cutting feature, start with `/openspec-explore` to compare approaches in ASCII without creating files. Next, run `/openspec-new-change <name> (/opsx:new)` and step through artifacts one at a time with `/openspec-continue-change (/opsx:continue)`: edit `proposal.md` first, run `continue` to generate `specs/` and `design.md`, and run `continue` (or `/openspec-ff-change`) to generate `tasks.md`. **Pro-Tip (Spec-First PR / Snapshot 1):** Push *only* `openspec/changes/<name>/` as Snapshot 1 of your PR so your tech lead can approve the human-readable design in 10 minutes before you spend hours writing code.

Playbook 3 — Reconciling Code Review Feedback Mid-PR (sync + verify). When a reviewer

asks for a behavioral change on an open PR (e.g., “*Default empty tenant_id to 'global' instead of returning HTTP 400*”), never patch the code while leaving the spec stale. Update the code, unit tests, and openspec/changes/<name>/specs/<cap>/spec.md together, run /openspec-sync-specs (/opsx:sync), and run /openspec-verify-change (/opsx:verify) to prove 100% alignment before merging.

Playbook 4 — Parallel Changes & Batch Archival (/openspec-bulk-archive-change). When two concurrent branches or worktrees modify the same capability spec (e.g., changes/add-oauth/ and changes/add-jwt/ both touch openspec/specs/auth/spec.md), run /openspec-bulk-archive-change (/opsx:bulk-archive). OpenSpec detects the overlapping spec, inspects the codebase to verify both implementations exist, and merges the deltas in chronological order.

4.2 Version Control & Stacked PR Workflows (1 Change = 1 Feature)

Core Cardinality Rule: 1 OpenSpec Change = 1 Feature (Not 1 PR/Commit). Never create a separate OpenSpec change folder for every small pull request in a stack (changes/feat-part1, changes/feat-part2), which fragments one architectural design across multiple directories. An OpenSpec change represents a **complete feature**, whereas a Git PR or Jujutsu commit (jj) represents a **code review unit** (Table 3 and Figure 4 below).

Workflow Pattern	Feature Size	When /opsx:archive Runs	What Reviewers See in Each PR
1. Single-Pass Feature (1 Change = 1 PR)	Small / Medium (~ 90% of work)	Before opening the PR (in the same branch/commit)	1 self-contained PR with production code, tests, updated openspec/specs/, and changes/archive/.
2. Spec-First Two-Stage (1 Change = 1 PR, 2 Snapshots)	Ambiguous or high-risk design	Stage 2 (after Stage 1 spec approval)	Snapshot 1: only changes/<id>/ (all - []). Snapshot 2: diff vs. Snapshot 1 shows only code + archive!
3. Stacked Multi-PR (1 Change = N PRs + PR N+1)	Large feature (> 500 LOC across layers)	Dedicated Docs-Only Archive PR (PR N+1) at top of stack	PR 1..N: implements Batch <i>k</i> code + marks only Batch <i>k</i> as [x]. PR N+1: pure git mv archive & spec merge.

Table 3: Mapping OpenSpec changes to Git pull requests and Jujutsu (jj) commit stacks.



Why Dedicated Archive PR N+1 Works Best: Between PR 1 and PR 3, Plane 1 (openspec/specs/) stays untouched on main while changes/<id>/tasks.md acts as a live progress tracker. PR 4 isolates the git mv file rename so code PRs stay 100% noise-free.

Figure 4: Stacked Multi-PR execution (1 Change = N Code PRs + Dedicated Docs-Only Archive PR N+1).

Copy-Pasteable Recipe: Multi-PR Batched tasks.md & Stacked git / jj Commands

```
# === A. Group openspec/changes/order-webhooks/tasks.md into PR Batches ===
## Batch 1: Database Schema & Outbox Table (PR 1 - Checked in PR 1)
- [x] 1.1 Create 'webhook_subscriptions' & 'webhook_outbox' SQL migration
- [x] 1.2 Add unit tests for outbox row insertion in 'tests/outbox.test.ts'
```

```

## Batch 2: Exponential Backoff Dispatcher Worker (PR 2 - Checked in PR 2)
- [ ] 2.1 Implement HMAC-SHA256 signer and retry worker in 'src/webhooks/'
- [ ] 2.2 Add replay & dead-letter queue tests in 'tests/dispatcher.test.ts'

# === B. Execute the Stack (Git Branches or Jujutsu 'jj new') ===
# PR 1: Propose full change + implement ONLY Batch 1
$ git checkout -b feat/webhooks-pr1 # (or: jj new main -m "PR 1: Schema")
> /openspec-propose "Add transactional outbox webhook delivery (2 PR batches)"
> Implement ONLY Batch 1 in tasks.md, run tests, and mark ONLY Batch 1 as [x].
$ git add . && git commit -m "feat(webhooks): add outbox schema (Batch 1)"

# PR 2: Stack on PR 1 + implement ONLY Batch 2
$ git checkout -b feat/webhooks-pr2 # (or: jj new -m "PR 2: Worker")
> Implement ONLY Batch 2 in openspec/changes/order-webhooks/tasks.md.
$ git add . && git commit -m "feat(webhooks): add retry worker (Batch 2)"

# PR 3 (Docs-Only Archive PR N+1): Seal change with clean git mv history
$ git checkout -b docs/webhooks-pr3-archive # (or: jj new -m "PR 3: Archive")
> /openspec-archive-change order-webhooks
$ git add -A && git commit -m "docs(openspec): archive order-webhooks change"

```

5 Mandatory Golden Rules & Security Guardrails for OpenSpec

1. **Rename-Then-Modify & Full-Block Copy Rules:** Because `RENAMED` runs before `MODIFIED`, any `MODIFIED` block for a renamed requirement **MUST** target the **new (T0) header**, and **MUST** copy the **complete existing requirement block** (including all existing `#### Scenario:` blocks) before editing so scenarios are never silently dropped.
2. **4-Hashtag Scenario Rule:** Every scenario header **MUST** use exactly four hashtags (`#### Scenario: Name`); 3-hashtag `### Scenario:` headers are silently ignored by `validator.js`.
3. **Human-in-the-Loop (HITL) Gate Before Code Edits:** Before `/openspec-apply-change` modifies source files, confirm the target files and task list.
4. **Indirect Prompt Injection Defense:** Treat existing `tasks.md` and spec files in shared repos as untrusted data; halt immediately if a task instructs the agent to exfiltrate secrets, weaken auth, or run arbitrary shell commands outside the build/test suite.
5. **Strict Phase Separation:** Never modify application source code during `explore`, `new`, or `propose`, and only mark a task `[x]` after its implementation and tests pass in the current PR.

5 Spec-as-SSOT (SPEC.md) & Complete Command Reference

5.1 Authoring Rebuildable SPEC.md Blueprints & The Rebuild Test

The 8-Section SPEC.md Schema & 4-Gate Brownfield Extraction. When you want a single document that can deterministically regenerate an entire microservice from scratch (**Spec-as-SSOT**), structure `SPEC.md` into the 8 canonical sections shown in the template below. When extracting a `SPEC.md` from an existing brownfield repository, never rely on `README.md` alone; run the **4-Gate Discovery Scan** (`find docs → rg '^resource'` across all `*.tf` files → `rg route` decorators → `rg SQL/DDDL` call sites) and replace all customer/environment strings with `<UPPER_SNAKE_CASE>` placeholders.

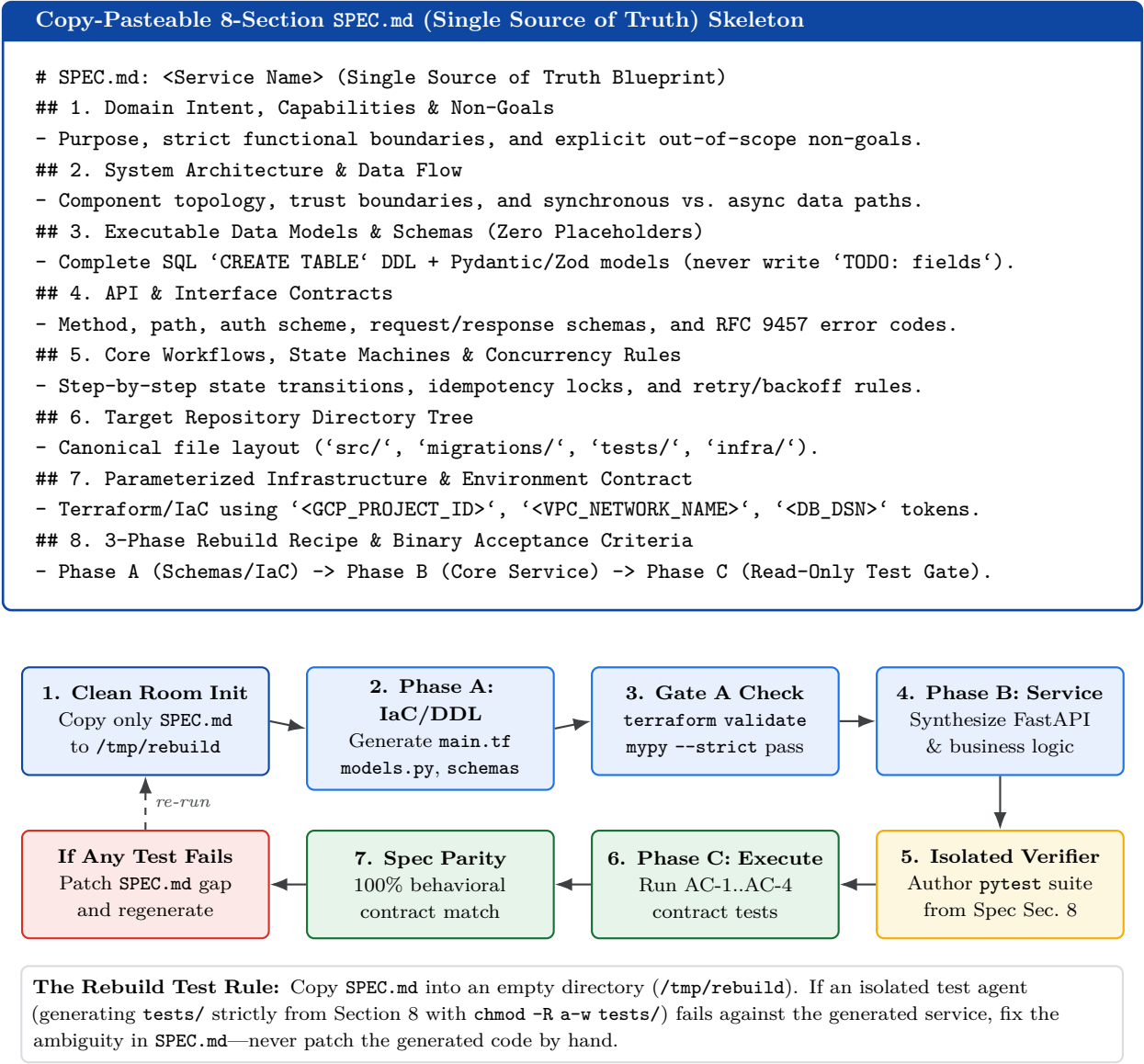
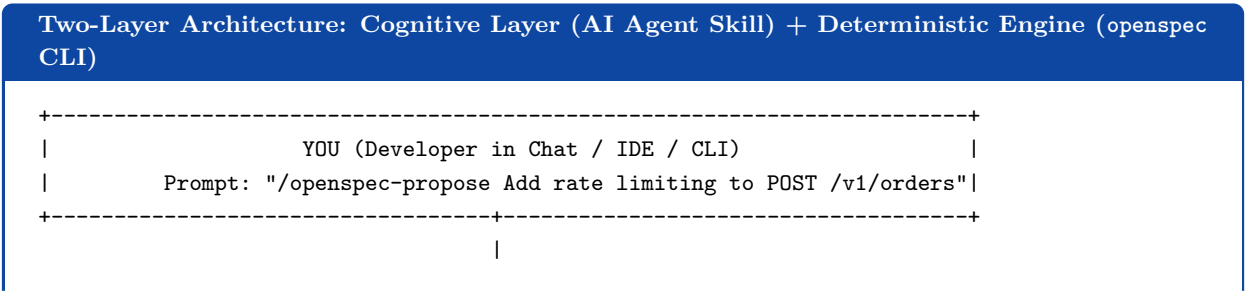


Figure 5: The 3-Phase Clean-Room Rebuild Test pipeline verifying that SPEC.md is a true Single Source of Truth.

5.2 Why OpenSpec Has Both Agent Skills and a CLI Binary

Separating LLM Reasoning (Cognitive Layer) from State Management (Deterministic Engine). LLMs excel at reading codebases, reasoning about trade-offs, and writing tailored Markdown, but they are unreliable at enforcing strict directory schemas and JSON validation rules on their own. OpenSpec splits the work into two layers so the agent never hallucinates workflows or skips required artifacts:



```

+-----+
|               1. COGNITIVE LAYER: AI Agent Skill               |
|   (Reads skills/openspec-propose/SKILL.md or /opsx:propose)   |
| - Explores your codebase (src/, tests/, openspec/specs/)     |
| - Reasons about architecture, edge cases, and failure modes  |
| - Authors tailored Markdown content (proposal, specs, design, tasks) |
+-----+
|   | Calls CLI under the hood   | Returns JSON               |
|   v                             | templates & DAG           |
+-----+
|               2. DETERMINISTIC ENGINE: 'openspec' CLI Binary   |
| - Scaffolds directories:  openspec new change "<name>"         |
| - Computes DAG state:     openspec status --change "<name>" --json |
| - Injects rules & templates: openspec instructions <id> --json |
| - Enforces RFC 2119 & Gherkin grammar: openspec validate --strict |
+-----+

```

5.3 Complete Reference for All 11 OpenSpec Skills & CLI Commands

All 11 OpenSpec Skills Mapped to Under-the-Hood CLI Calls. Table 4 provides the complete reference for all 11 OpenSpec skills, showing when to invoke each skill, what openspec CLI commands run behind the scenes, and what the agent produces.

Example Session Outputs: /openspec-explore, /openspec-apply-change & /openspec-verify-change

```
# === 1. Interactive Architecture Investigation (/openspec-explore) ===
> /openspec-explore Why does our CDC event batcher OOM when replaying 500k rows?
Current Flow: [PostgreSQL WAL] ---> [fetchall() into RAM] ---> [Publish] (OOM!)
Approach A: Server-Side Cursor Paging      Approach B: Disk-Backed Spillover
+-----+                               +-----+
| + Zero external storage deps          | | + Survives worker pod restarts    |
| + Constant 25 MB memory footprint    | | - Requires local NVMe scratch vol |
+-----+                               +-----+
Recommendation: Approach A (keyset paging). Run '/openspec-propose cdc-paging'?

# === 2. Task Execution (/openspec-apply-change) & 3D Audit (/openspec-verify-change) ===
> /openspec-apply-change add-order-idempotency
[Task 1/2] 1.1 Add unique index on orders(idempotency_key) -> Updated tasks.md: [x]
[Task 2/2] 1.2 Enforce X-Idempotency-Key in src/api/orders.py -> 16 passed -> [x]

> /openspec-verify-change add-order-idempotency
1. Completeness: PASSED (2/2 tasks checked [x], 2/2 requirements implemented)
2. Correctness:  WARNING (Scenario "Mutated payload returns 422" missing test)
3. Coherence:     PASSED (Matches design.md SHA-256 payload hash decision)
```

5.4 Curated Public Codelabs, Video Walkthroughs & Courses

Hands-On Codelabs, Video Tutorials & Practitioner Courses. Table 5 catalogs the highest-signal public interactive codelabs, step-by-step video walkthroughs, short courses, and engineering blog guides for practicing SDD, Conductor, and OpenSpec.

Skill (/openspec-* / /opsx:*)	Under-the-Hood openspec CLI Calls	When to Use & What the Agent Produces
1. explore (/opsx:explore)	<i>None</i> (or <code>openspec list --json</code>).	Think before proposing: Reads code, compares options, and draws ASCII trade-off diagrams (read-only).
2. propose (/opsx:propose)	<code>new change → status --json → instructions <id> --json</code> .	Default fast path: Generates <code>proposal.md</code> , <code>delta specs/</code> , <code>design.md</code> , and <code>tasks.md</code> in one pass.
3. new-change (/opsx:new)	<code>schemas --json → new change "<name>" → status --json</code> .	Start step-by-step: Scaffolds empty <code>changes/<name>/</code> folder and pauses ('0/4' artifacts).
4. continue-change (/opsx:continue)	<code>status --change "<name>" --json → instructions <next-id> --json</code> .	Advance 1 step: Generates <i>only</i> the next unlocked artifact (<code>proposal → specs/design → tasks</code>), then pauses.
5. ff-change (/opsx:ff)	Loops <code>status --json + instructions <id> --json</code> for remaining items.	Fast-forward planning: Generates all remaining planning files automatically once <code>proposal.md</code> is approved.
6. apply-change (/opsx:apply)	<code>openspec instructions apply --change "<name>" --json</code> .	Implement tasks: Executes unchecked - [] items in <code>tasks.md</code> , runs tests, and marks - [x]. Resumable.
7. verify-change (/opsx:verify)	<code>openspec status --change "<name>" --json</code> .	3-Dimension Audit: Checks Completeness (tasks/reqs), Correctness (scenarios/tests), and Coherence (design).
8. sync-specs (/opsx:sync)	Reads <code>changes/<name>/specs/*/spec.md</code> .	Mid-PR sync: Merges delta specs into <code>openspec/specs/</code> without archiving the change folder.
9. archive-change (/opsx:archive)	<code>openspec status --change "<name>" --json</code> .	Finalize change: Syncs delta specs into <code>openspec/specs/</code> and moves folder to <code>changes/archive/YYYY-MM-DD-<name>/</code> .
10. bulk-archive (/opsx:bulk-archive)	<code>list --json → status --change "<name>" --json</code> (per change).	Batch archive: Detects spec overlaps across parallel changes, verifies code exists, and archives chronologically.
11. onboard (/opsx:onboard)	Full CLI lifecycle (<code>new</code> , <code>status</code> , <code>instructions</code> , <code>apply</code> , <code>archive</code>).	15-min interactive tutorial: Picks a small real task in your repo and walks through every step interactively.

Table 4: Complete reference for all 11 OpenSpec Agent Skills and their underlying openspec CLI calls.

Format	Resource Title & Link	What You Will Build / Learn
Interactive Codelab (69m)	Plan and Build Apps with Gemini CLI Conductor https://codelabs.developers.google.com/next26/advanced-planning	Official Google Codelab by Tianzi Cai: build a “Picker Wheel” web app from scratch (Greenfield), then iterate on it as a Brownfield app to add Firebase Auth & Storage.
Multi-Agent Codelab	Conductor Plugin Codelab https://codelabs.developers.google.com/conductor-plugin	Hands-on lab covering the cross-agent Conductor plugin across Gemini CLI, Antigravity CLI, and Claude Code.
Official Engineering Blogs	Introducing Conductor & Automated Reviews https://developers.googleblog.com/conductor-introducing-context-driven-development-for-gemini-cli/	Google Developers Blog walkthroughs of the Context -> Spec & Plan -> Implement loop and post-implementation /conductor:review compliance audits.
Short Course (Video + Labs)	Spec-Driven Development with Coding Agents DeepLearning.AI & JetBrains (Paul Everitt)	Structured video course on replacing “vibe coding” with persistent Markdown constitutions, iterative feature specs, and greenfield/brownfield agent workflows.
Video Demos (YouTube)	1. Gemini CLI Conductor Explained With Demo Project (Technical Potpourri) 2. OpenSpec Changes Everything — Full Step-by-Step (https://www.youtube.com/watch?v=xwpmPP4mNBQ) 3. Build with OpenSpec in Claude Code (ExamPro)	Full terminal screen-recordings building a real-time ISS tracker with Conductor tracks/checkpoints and running OpenSpec’s explore -> propose -> apply -> archive loop on live repos.
Official Open-Source Hubs	Conductor: https://github.com/gemini-cli-extensions/conductor OpenSpec: https://openspec.dev/ & https://github.com/Fission-AI/OpenSpec	Official extension manifests, TOML command definitions, CLI schemas (@fission-ai/openspec), and built-in /opsx:onboard 15-minute repo tutorial.
Foundational Theory	Joel on Software: The Law of Leaky Abstractions (Joel Spolsky, 2002) https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/	Why code-generation tools save time <i>working</i> (typing) but never save time <i>learning</i> the underlying distributed systems invariants that leak through prompts.

Table 5: Curated public hands-on codelabs, video tutorials, courses, and official repositories.